Hidden in plainsight

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

You're given a seemingly ordinary JPG image. Something is tucked away out of sight inside the file. Your task is to discover the hidden payload and extract the flag.

💡 Hints

- Download the jpg image and read its metadata

🧰 Tools Used

- **file**: Utility to identify file types and properties
- **strings**: Command-line tool to extract readable ASCII sequences from files
- **grep**: Text search utility for filtering output
- **exiftool**: Metadata inspection and manipulation tool
- **base64**: Encoding/decoding utility for Base64 data
- **steghide**: Steganography tool for extracting hidden data from images
- **xxd**: Hexadecimal viewer and editor

🔍 Complete Solution

Step 1: Download and Prepare

Download the file named `img.jpg` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file img.jpg
```

Expected output:

```
img.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, comment: "c3RlZ2hpZGU6Y0VGNmVuZHZjbVE9", baseline, precision 8, 640x640, components 3
```

Step 3: Search for Visible Text

The flag may be present as plain text. Try extracting all character strings:

```
strings img.jpg | grep -i "picoCTF"
```

⚠️ *If no results appear, proceed to the next step (metadata).*

Step 4: Inspect Metadata

This is the crucial step! Use `exiftool` to examine all image metadata:

```
exiftool img.jpg
```

Complete output:

```
ExifTool Version Number         : 13.50
File Name                       : img.jpg
Directory                       : .
File Size                       : 74 kB
File Modification Date/Time     : 2025:11:07 21:17:32+02:00
File Access Date/Time           : 2026:05:05 10:48:30+02:00
File Inode Change Date/Time     : 2026:05:05 10:47:16+02:00
File Permissions                : -rw-r--r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Comment                         : c3RlZ2hpZGU6Y0VGNmVuZHZjbVE9
Image Width                     : 640
Image Height                    : 640
Encoding Process                : Baseline DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 640x640
Megapixels                      : 0.410
```

🎯 **Key point**: Notice the `Comment` field which contains a suspicious, encoded string!

Step 5: Decode the Base64 Value

The string in the `Comment` field is encoded in Base64:

```
c3RlZ2hpZGU6Y0VGNmVuZHZjbVE9
```

Decode it using one of these commands:

```
echo 'c3RlZ2hpZGU6Y0VGNmVuZHZjbVE9' | base64 --decode
```

Decoded result:

```
steghide:cEF6endvcmQ=
```

This indicates that the image contains hidden data using `steghide`, and the password is encoded in the next part.

Step 6: Decode the Password

The password part `cEF6endvcmQ=` is also Base64 encoded:

```
echo 'cEF6endvcmQ=' | base64 --decode
```

Decoded password:

```
pAzzword
```

Step 7: Extract Hidden Data with Steghide

Now, use `steghide` to extract the hidden content from the image using the decoded password:

```
steghide extract -sf img.jpg -p pAzzword
```

This will extract a file (likely containing the flag). Check the extracted file for the flag.

```
cat flag.txt
```

✅ Final Result

The extracted file contains the flag:

```
picoCTF{<REDACTED>}
```

🧠 Key Concepts

- **Metadata**: Hidden information in files (comments, EXIF data, etc.)
- **Base64**: Common encoding to obscure or compress content
- **Steganography**: Hiding data within other data (like images)
- **Anomaly Recognition**: Looking for elements that seem suspicious or out of place